Tool Lifecycle for Agentic Systems

Framework note — Volume 5

2026-04-17

Contents

The Brain Layer **2**

1. Problem . . . 2
2. Definition . . . 2
3. Responsibilities . . . 3
4. Design principles . . . 3
5. Minimum viable brain layer . . . 3
6. What it buys you . . . 4
7. Anti-patterns to avoid . . . 4
8. Companion documents . . . 4

Query Classification for Multi-Mode Reasoning **4**

1. Why classify? . . . 4
2. Five canonical modes . . . 4
3. Classifier design . . . 5
4. Confidence and override . . . 5
5. Registering a new mode . . . 5
6. Anti-patterns . . . 6
7. Observability . . . 6

The Shared-Data Layer **6**

1. Why . . . 6
2. Properties . . . 6
3. Architecture . . . 7
4. Read discipline . . . 7
5. Write discipline . . . 7
6. Provenance . . . 8
7. Common failure modes . . . 8
8. What the shared-data layer does *not* do . . . 8

Role-Conditioned Rendering **8**

1. Principle . . . 8
2. What changes, what doesn't . . . 8
3. The role contract . . . 9
4. Renderer architecture . . . 9

5. Rules of thumb 9
6. Anti-patterns 10
7. Adding a role 10
8. Retiring a role 10

Tool Lifecycle for Agentic Systems **10**
1. Why a lifecycle 10
2. The six stages 11
3. Metadata every tool carries 11
4. Stage gates 11
5. The review cadence 12
6. What disqualifies a tool from `stable` 12
7. Tool hygiene signals 12
8. The tombstone 13

The Brain Layer

1. Problem

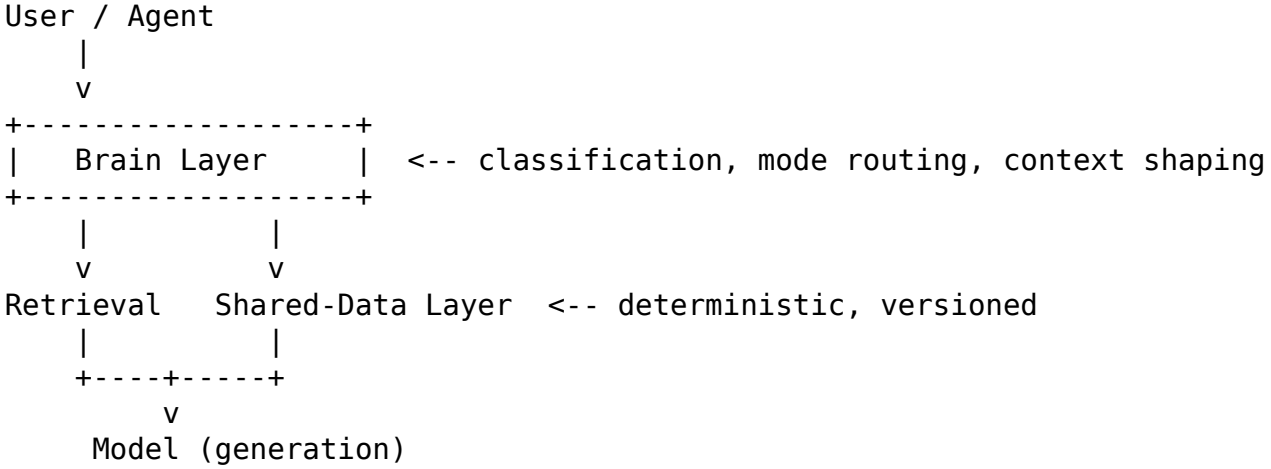
Large language models are powerful generalists, but agentic systems that answer high-stakes questions — regulatory, operational, strategic — need more than raw generation. They need:

- **Deterministic ground truth** that does not drift between sessions.
- **Structured reasoning modes** so the system knows *how* to answer, not just *what* to say.
- **Role-appropriate output** so the same fact set can serve different operators.
- **Auditable provenance** so every claim can be traced to a source.

Model-only pipelines fail on all four. The **brain layer** pattern closes the gap.

2. Definition

The brain layer is a **logic tier placed between retrieval and generation**. It classifies the incoming question, selects an appropriate reasoning mode, pulls deterministic context from a shared-data layer, and shapes the model's final response.



```
    |
    v
Role-conditioned renderer
    |
    v
  Output
```

3. Responsibilities

The brain layer owns five responsibilities:

1. **Classify the query.** Determine whether the question is factual, causal, comparative, temporal, or cross-domain. Classification is a cheap, deterministic step.
2. **Select a reasoning mode.** Each class has a matching mode with its own prompt scaffold, context budget, and evidence requirement.
3. **Fetch deterministic context.** Pull structured rows from the shared-data layer; never rely on the model to "remember."
4. **Shape the model prompt.** Inject the retrieved context, the mode scaffold, and any role constraints. Keep the prompt surface small and predictable.
5. **Emit a structured response.** The output is a typed object — summary, evidence list, confidence, open items — not free text.

4. Design principles

- **Determinism first.** Any step that *can* be deterministic *must* be. Use the model only where creativity or synthesis is required.
- **Single source of truth.** Shared-data layer is the one place facts live. Caches are derived; prompts are derived; renderers are derived.
- **Mode-per-intent.** Different question shapes deserve different machinery. A comparison question should not run through a causal pipeline.
- **Small, typed contracts.** Every brain-layer call produces a schema-validated object. Drift is caught at the boundary, not in the UI.
- **Least-privilege retrieval.** Read paths run read-only; write paths are isolated, authenticated, and rate-limited.

5. Minimum viable brain layer

A working brain layer needs four components:

Component	Role
Classifier	Labels the query with a reasoning mode
Retriever	Pulls deterministic rows from the shared-data layer
Prompt shaper	Combines mode scaffold + context + role constraints
Response contract	Validates and normalizes the model output

None of these are the model. The model is a tool the brain layer calls — not the brain itself.

6. What it buys you

- **Consistency.** The same question produces the same shape of answer every time.
- **Auditability.** Every claim has a traceable row-level source.
- **Swappable models.** Because the brain layer owns the contracts, the generation model becomes a replaceable component.
- **Controlled blast radius.** A bad model response cannot corrupt ground truth; it can only fail validation.

7. Anti-patterns to avoid

- Treating the model as the brain. The model is a renderer, not a reasoner.
- Letting retrieval happen inside the prompt ("search the web and answer"). Retrieval belongs above the model, not inside it.
- Free-text outputs. Always emit typed, validated objects.
- Letting one mode fit all questions. A single prompt scaffold cannot serve causal, comparative, and temporal intents well.

8. Companion documents

- **Query Classification for Multi-Mode Reasoning** — how the classifier works
- **Shared-Data Layer for Agent Coordination** — how deterministic context is stored and served
- **Role-Conditioned Rendering** — how the same facts produce different views per operator

Query Classification for Multi-Mode Reasoning

1. Why classify?

Not all questions want the same answer shape. A causal question expects a cause-effect chain. A comparative question expects a side-by-side. A temporal question expects a timeline. Running every question through a single prompt produces mush.

A **query classifier** is a small, deterministic step that labels the incoming question with a reasoning mode, so the brain layer can route it to the right scaffold.

2. Five canonical modes

This taxonomy covers the majority of high-stakes questions agentic systems encounter.

2.1 Factual (auto)

- **Intent:** retrieve and present a known fact.
- **Trigger words:** *what is, list, show, define, status of.*
- **Output shape:** summary + evidence rows.

2.2 Causal

- **Intent:** explain why something happened, or what drives an outcome.
- **Trigger words:** *why, because, drove, caused, led to, root cause.*
- **Output shape:** cause->effect chain, supporting rows, confidence.

2.3 Contrast

- **Intent:** compare two or more entities side-by-side.
- **Trigger words:** *vs, compared to, difference between, which is stricter.*
- **Output shape:** aligned table with columns per entity.

2.4 Temporal

- **Intent:** track change over time, or compare versions.
- **Trigger words:** *trend, history, since, over time, version, changed.*
- **Output shape:** ordered timeline with deltas.

2.5 Cross-domain

- **Intent:** transfer an answer across domains, jurisdictions, or contexts.
- **Trigger words:** *elsewhere, in other states, applies to, equivalent, map to.*
- **Output shape:** base entity + mapped entity + delta report.

3. Classifier design

A good classifier is **cheap, deterministic, and transparent.** Three layers, in priority order:

1. **Pattern rules.** Regex / keyword matchers tied to each mode. Hit rate: 60-80%.
2. **Structural hints.** Request metadata (e.g., `compare_with` field present -> contrast; `since` parameter present -> temporal). Hit rate: +10-15%.
3. **Model fallback.** Only when rules and structure agree on nothing. Small, cheap model call with a constrained output: one of the five mode labels. Hit rate: remainder.

Do not start with a model call. Model-first classifiers are slower, less stable, and harder to debug.

4. Confidence and override

The classifier emits `{ mode, confidence, alternates[] }`. Downstream code can:

- Accept the top mode if confidence >= threshold.
- Run two modes in parallel if the top two are close and reconcile outputs.
- Fall back to factual mode when confidence is low — factual is the safest default.

5. Registering a new mode

Adding a sixth mode is a four-step change:

1. Define the intent and output shape.
2. Add pattern rules and structural hints.
3. Write the prompt scaffold and response contract.
4. Wire the mode into the brain-layer router.

The new mode should earn its keep: if its questions can be handled by an existing mode with a minor scaffold tweak, do that instead. Mode proliferation erodes the taxonomy.

6. Anti-patterns

- **One prompt to rule them all.** If your prompt has "if the user is asking about causes, do X; if comparing, do Y," you do not have a prompt — you have an unextracted classifier.
- **LLM-first classification.** Every classification call is a latency and cost hit. Rules and structure first.
- **Silent reclassification.** The classifier's output belongs in the response metadata so the caller knows how the question was interpreted.
- **Unbounded alternates.** Cap `alternates[]` at two; more than that is noise.

7. Observability

Every classification emits a log line with `{ query_hash, mode, confidence, source }` where `source` is `rules | structure | model`. Reviewing this log weekly tells you:

- Which modes are actually used.
- Where the classifier is forced into model fallback.
- Where new pattern rules should be added to reduce fallback cost.

The Shared-Data Layer

1. Why

Agents, classifiers, renderers, and schedulers all need the same facts. If each component carries its own copy, they drift. If each component fetches fresh from the internet, they're slow, inconsistent, and non-reproducible. The **shared-data layer** is the one place facts live, and every other component reads from it.

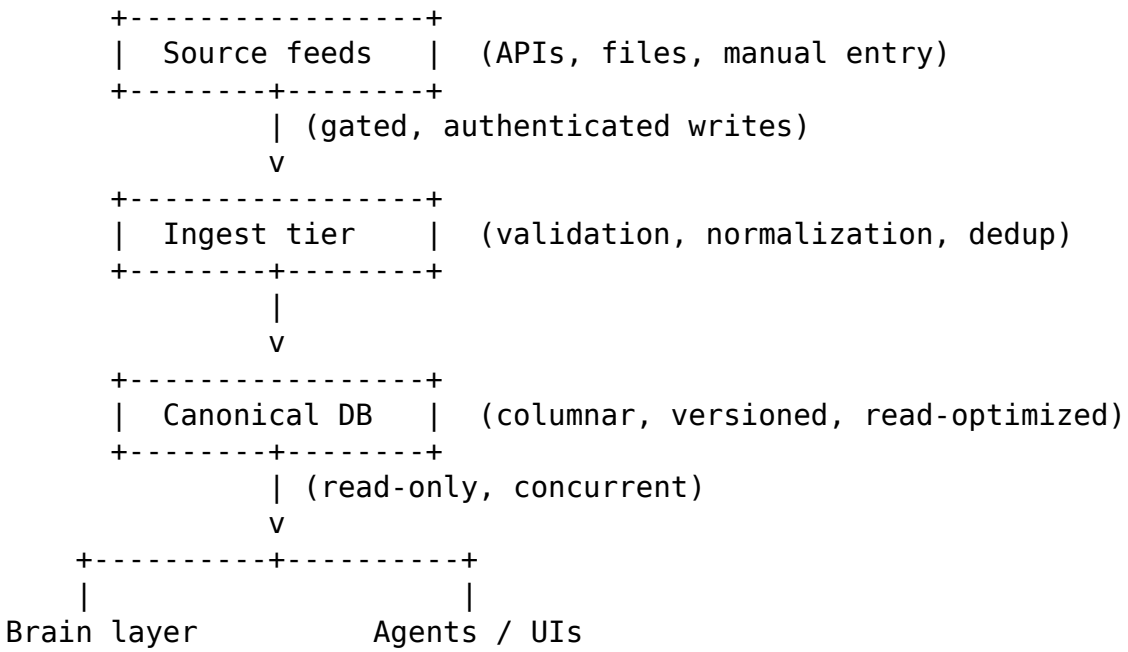
2. Properties

A shared-data layer for agentic systems has six non-negotiable properties:

1. **Deterministic.** Same query, same result. No nondeterminism from model calls or time-of-day drift.
2. **Versioned.** Every row has a source, a captured-at timestamp, and a content hash.
3. **Read-fast, write-gated.** Readers are concurrent and cheap. Writers are authenticated, rate-limited, and observed.
4. **Schema-enforced.** Every table has a declared schema; inserts that don't match are rejected at the boundary.

5. **Queryable in SQL.** Structured, columnar, indexable. Not a document store. Not a vector store (those are adjunct, not primary).
6. **Local-first.** The layer lives on the operator's infrastructure. External services are sources, not stores.

3. Architecture



4. Read discipline

- **Read-only subprocess by default.** If the query does not write, the process should not be able to.
- **Bounded execution.** Every query has a timeout. A runaway query cannot stall the fleet.
- **Parameterized everything.** No user-controlled string concatenation in SQL. Bind values; escape them at one chokepoint.
- **Result caps.** Every read path has a row cap appropriate to the caller (UI: small, agent: larger, export: bounded).
- **Caching.** A short TTL cache sits in front of the canonical DB. Invalidate on write; bypass for freshness-critical reads.

5. Write discipline

- **Single writer pattern.** One ingest tier, one merge lock, no concurrent writes against the canonical DB.
- **Authenticated.** Writes require a bearer token tied to the writer identity.
- **Pre-validated.** Schema check happens in the ingest tier, not at the DB. Bad rows never hit the canonical store.
- **Append-preferred.** Favor append + merge over in-place update. History becomes a first-class asset.

- **Logged.** Every write emits `{ actor, table, row_count, content_hash, ts }` to an audit log.

6. Provenance

Every row carries:

- `source_id` — which feed it came from
- `captured_at` — when the feed delivered it
- `content_hash` — stable hash of the row content, for dedup and diffing
- `version` — monotonic per-entity version counter

This is what makes the system auditable. "Where did this claim come from" is a SQL join, not a guess.

7. Common failure modes

- **Writer sneaks a read client into the write process.** Now you can't bound blast radius. Split them.
- **Schema drift.** A source adds a field; downstream breaks. Solution: a schema-sentry job that diffs source vs. declared schema and alerts.
- **Cache staleness.** A write happens but readers keep seeing stale rows. Solution: cache key includes latest version; write bumps version.
- **Lock contention.** Multiple writers compete; reads stall behind them. Solution: enforce single-writer, move heavy writes to a background queue.

8. What the shared-data layer does *not* do

- It does **not** do reasoning. Reasoning lives in the brain layer above it.
- It does **not** do presentation. Presentation lives in role-conditioned renderers below it.
- It does **not** do retrieval ranking. Ranking belongs to the retriever or brain layer.

Keep it boring. Boring storage is reliable storage.

Role-Conditioned Rendering

1. Principle

The same fact set serves different operators in different ways. A field inspector wants a checklist. A senior reviewer wants risk signals. An executive wants a single line. The facts do not change. The *rendering* does.

Role-conditioned rendering is the pattern that separates facts from presentation, and lets the same brain-layer response flow into multiple operator-specific views without re-querying.

2. What changes, what doesn't

Concern	Varies by role?
Ground-truth facts	No
Evidence rows	No
Confidence scores	No
Which fields are surfaced	Yes
Ordering / priority	Yes
Language density (terse vs. expanded)	Yes
Recommended action	Yes
Visual format (list, table, card, narrative)	Yes

If a role needs *different facts*, that is not rendering — that is a different query. Keep the boundary clean.

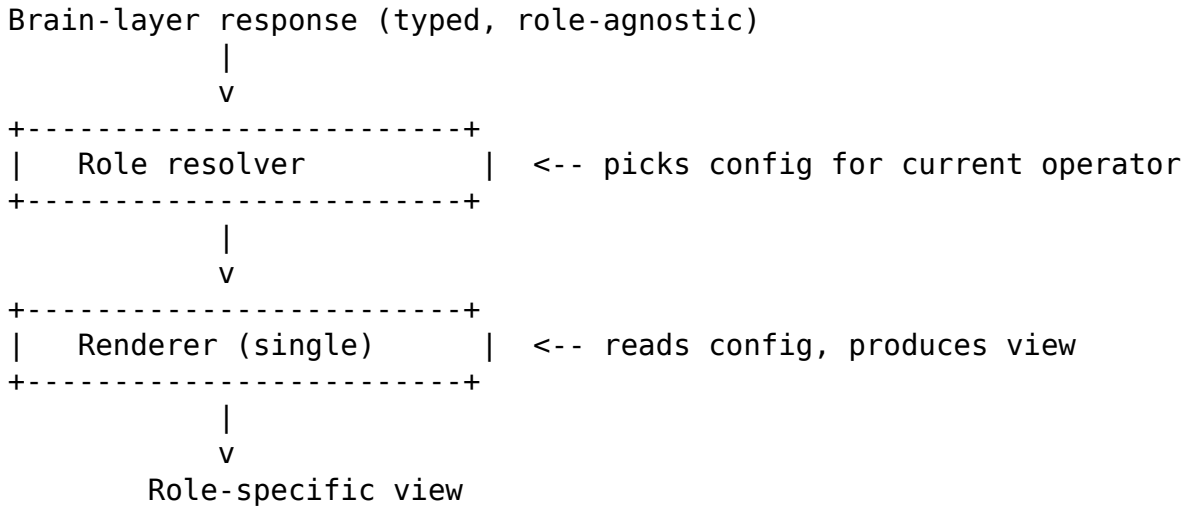
3. The role contract

Every role is declared as a small config object:

```
role: inspector
fields_surfaced: [status, next_action, blocking_items]
language: terse
action_cta: "Mark pass / fail / hold"
format: checklist
priority_order: [blocking_items, status, everything_else]
```

New roles are new configs. The renderer is one piece of code; role configs are data.

4. Renderer architecture



One renderer, many configs. Do not fork the renderer per role.

5. Rules of thumb

- **No role secrets.** Every role sees the same truth. Role-specific *hiding* should be rare and documented; role-specific *emphasis* is the normal case.

- **Language density is a dial, not a fork.** Terse inspector language and expanded reviewer language come from the same sentence generator with a length constraint.
- **Action CTAs belong in the role config.** The brain layer does not know what button to show; the renderer does, conditioned on role.
- **Log what each role saw.** When someone asks "why did my inspector miss this," you want to answer with the exact rendered view, not guess.

6. Anti-patterns

- **Duplicated renderers.** If you have `renderForInspector()` and `renderForReviewer()` as parallel code paths, you will drift. Collapse them into `render(response, roleConfig)`.
- **Role leak into the brain layer.** If the brain layer branches on role to change retrieval, that is the wrong layer. Roles shape presentation; they do not shape truth.
- **Secret data hidden in one role's view but present in the response payload.** That is a leak waiting to happen. If a role should not see it, it should not be in the response.
- **Unbounded roles.** Every new role costs review and QA. Add them deliberately. Retire unused ones.

7. Adding a role

1. Draft the role config.
2. Decide if any new fields are needed in the brain-layer response. If yes, that's a response-contract change — coordinate upstream.
3. Write a view test that asserts a canonical response renders the expected view.
4. Ship the config. The renderer needs no changes.

8. Retiring a role

Roles accumulate. Retire them when:

- No operator has used the role for a quarter.
- The role's config duplicates an existing role with minor tweaks (merge them).
- The role's purpose is absorbed by a different workflow.

Retirement is a deletion of the config plus a changelog entry. No renderer code changes.

Tool Lifecycle for Agentic Systems

1. Why a lifecycle

Agentic systems accumulate tools faster than they retire them. A healthy tool library has discipline: each tool has a visible stage, an owner, a last-reviewed date, and a clear exit path. Without that, the library silently fills with half-built, half-forgotten capabilities that drag on the whole system's reliability.

2. The six stages

Stage	Meaning	Visible to agents?
`proposed`	Specification exists, no implementation	No
`alpha`	Working prototype, narrow use only	Gated
`beta`	Broader internal use, still evolving	Gated
`stable`	Production-ready, contract locked	Yes
`deprecated`	Works, but a successor exists	Yes, with warning
`retired`	Removed from the library	No

A tool moves forward through these stages, or it gets retired. It does not sit in `alpha` forever.

3. Metadata every tool carries

Every tool ships with a small manifest:

```
name: example-tool
stage: beta
owner: <team or person>
created: 2026-01-15
last_reviewed: 2026-04-10
successor: null
description: One-sentence purpose statement.
contract: reference to the input/output schema
```

The manifest is a first-class artifact. It is read by the system that lists tools to agents, and it is reviewed on a schedule.

4. Stage gates

Each transition has a gate. You do not move forward by wishing.

proposed -> alpha

- Implementation exists.
- Smoke test passes.
- At least one caller has exercised it end-to-end.

alpha -> beta

- Input / output contract is declared and validated.
- Error paths return typed errors, not raw exceptions.
- A test harness covers the top three call patterns.

beta -> stable

- Contract has been stable for 30+ days without breaking changes.
- Observability is in place (request log, error rate).
- A rollback path exists (previous version, or a disable flag).

stable -> deprecated

- A successor tool is available and at least `beta`.
- A migration note exists telling callers what to do.
- A sunset date is set (typically 30-90 days out).

deprecated -> retired

- All known callers have migrated, or the sunset date has passed.
- One final notification goes out (or a last log entry is written).
- Code and manifest are removed; a tombstone entry remains in the lifecycle log.

5. The review cadence

The tool library gets reviewed on a cadence. A good rhythm:

- **Weekly:** scan for any tool with errors in the last 7 days. Triage.
- **Monthly:** review every tool's `last_reviewed` date. Anything older than 90 days either gets reviewed and renewed, or moves toward deprecation.
- **Quarterly:** audit the retirement candidates. Remove anything that has not earned its slot.

The cadence is the thing. A library with no review process has no discipline.

6. What disqualifies a tool from `stable`

- Hardcoded secrets, credentials, or environment assumptions.
- String concatenation into downstream systems without sanitization.
- No timeout on subprocess calls.
- No authentication on a tool that produces writes or side effects.
- Error messages that leak internal state to callers.
- A contract that has changed more than twice in the last 30 days.

If any of these is true, the tool is at most `beta`.

7. Tool hygiene signals

Healthy library:

- Mean stage is `stable` or `beta`.
- Number of tools `alpha` or `proposed` is bounded.
- Quarterly retirement list is non-empty — the library is actually shrinking things.
- Time-from-`alpha`-to-`stable` is measured and decreasing over time.

Unhealthy library:

- Tools stuck in `alpha` for months.
- More tools added than retired each quarter.
- No tool has ever been retired.
- Tools without `last_reviewed` dates.

8. The tombstone

Retired tools leave a one-line entry in a retirement log:

```
2026-04-17  example-tool  retired  superseded by example-tool-v2
```

This is the memory. When a new caller wonders “did we ever have a tool for X?”, the retirement log is where they look first. Never delete the log.